Session 1 - Introduction to Java

What is Java ?

- Java is a programming language
- Java developed by James Gosling & his team in 1991 at Sun Microsystems
- Initially they named it as `OAK` programming language
- In 1995, OAK language renamed to Java
- Oracle Corporation acquired Sun Microsystem in 2010
- Now java is under license of Oracle corporation
- Java is a free & open-source software
- 3 billion devices run on Java language
- Java is one of the most popular languages in the world
- Java can run on any platform (It is platform independent)
- Latest version of Java is Java 25 (LTS)

Java Features

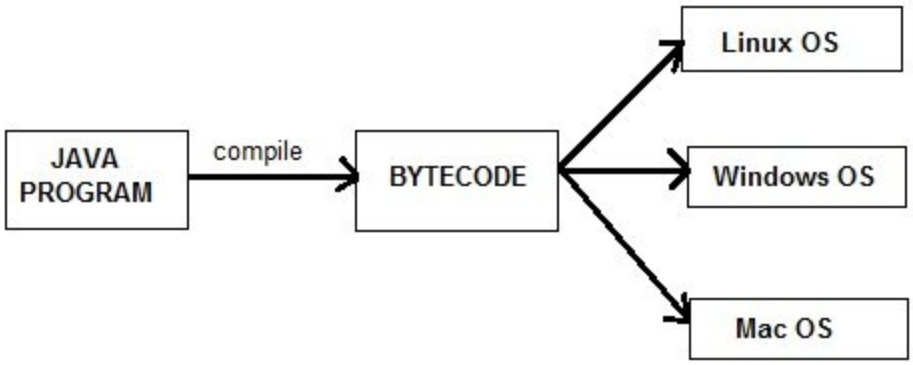
Simple

- Java is easy to learn and its syntax is quite simple, clean and easy to understand. The confusing and ambiguous concepts of C++ are either left out in Java or they have been re-implemented in a cleaner way.

```
Eg: Pointers and Operator Overloading are not there in java
```

Platform Independent

- Unlike other programming languages such as C, C++ etc which are compiled into platform specific machines. Java is guaranteed to be write-once, run-anywhere language.
- When we compile java code it will generate bytecode. This bytecode is platform independent and can be run on any machine, plus this bytecode format also provide security. Any machine with Java Runtime Environment can run Java Programs.



OOP Language

- Java is an object-oriented programming language. Everything in Java is an object. Object-oriented means we organize our software as a combination of different types of objects that incorporate both data and behaviour.
- Object-oriented programming (OOPs) is a methodology that simplifies software development and maintenance by providing some rules.
- OOPs Principles are:

```
Object
Class
Inheritance
Polymorphism
Abstraction
Encapsulation
```

Secure

- When it comes to security, Java is always the first choice. With java secure features it enables us to develop virus free, temper free system. Java program always runs in Java runtime environment with almost null interaction with system OS, hence it is more secure.

Multi-Threading

- Java multithreading feature makes it possible to write program that can do many tasks simultaneously. Benefit of multithreading is that it utilizes same memory and other resources to execute multiple threads at the same time, like While typing, grammatical errors are checked along.

Architectural Neutral

- Compiler generates bytecodes, which have nothing to do with a particular computer architecture, hence a Java program is easy to interpret on any machine.

Portable

- Java Byte code can be carried to any platform. No implementation dependent features. Everything related to storage is predefined.

```
Eg: size of primitive data types
```

High Performance

- Java is an interpreted language, so it will never be as fast as a compiled language like C or C++. But Java enables high performance with the use of just-in-time compiler.

Distributed

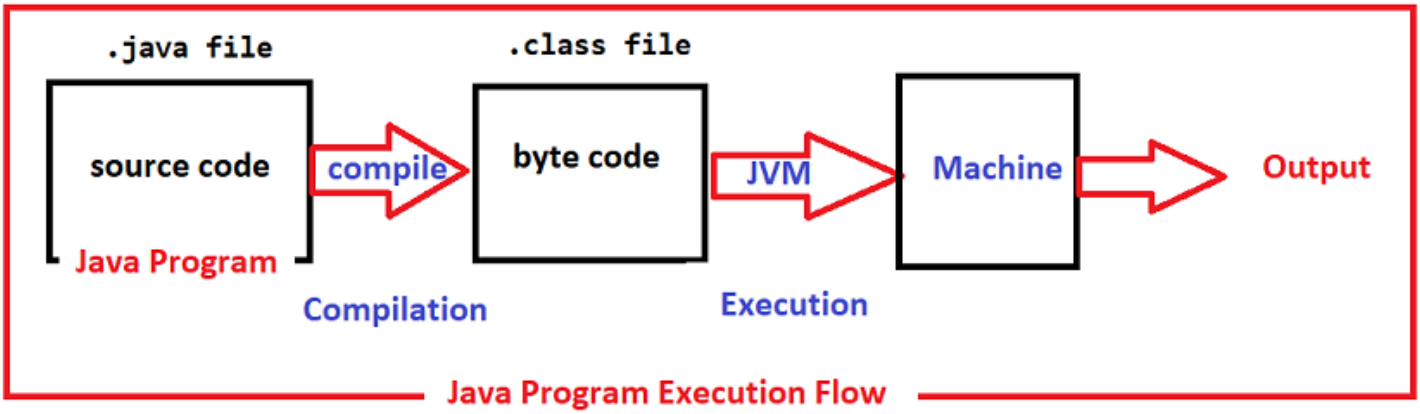
- Java is also a distributed language. Programs can be designed to run on computer networks. Java has a special class library for communicating using TCP/IP protocols. Creating network connections is very much easy in Java as compared to C/C++.

Difference between JDK, JRE & JVM ?

- JDK contains set of tools to develop java programs
- JRE providing a platform to run our java programs
- JVM will take care of program execution

Java program Execution Flow

Step-1: We will write source code and we will save that code in a file using `.java` extension
Step-2: We will compile source code using java compiler (it will generate byte code)
Step-3: We will execute `.class` file (JVM will convert byte code into machine code & gives output)



Bytecode

- The output of a Java compiler is not executable code. Rather, it is bytecode
- Bytecode is a highly optimized set of instructions designed to be executed by the Java run-time system
- Bytecode makes Java platform-independent

JVM

- JVM stands for Java Virtual Machine (We can't see with our eyes)
- JVM will be part of JRE
- JVM is responsible for executing java programs
- JVM will allocate memory required for program execution & de-allocate memory when it is not used
- JVM will convert byte code into machine understandable format

First Java Program Structure

```
Package statement
Import statement
Class
Variable
Methods
```

- Package statement is used to group our classes. At most we can write only one package statement in one java program and package statement should be the first statement in the program.

- Import statements are used to import one program into another program. We can write multiple import statements in one program (It depends on requirement). We should write import statements after package statement only.
- Class is the most import part in java program. Class is a plan to define program elements. Inside class we will write variables and methods.
- Variables are used to store the data. We can write any no. of variables inside a class.
- Methods are used to perform action. Application business logic we will write inside methods. A can class can have any no. of methods.

```
class Test {
        public static void main(String... args){
                System.out.println("Welcome to Java !!");
        }
}
```

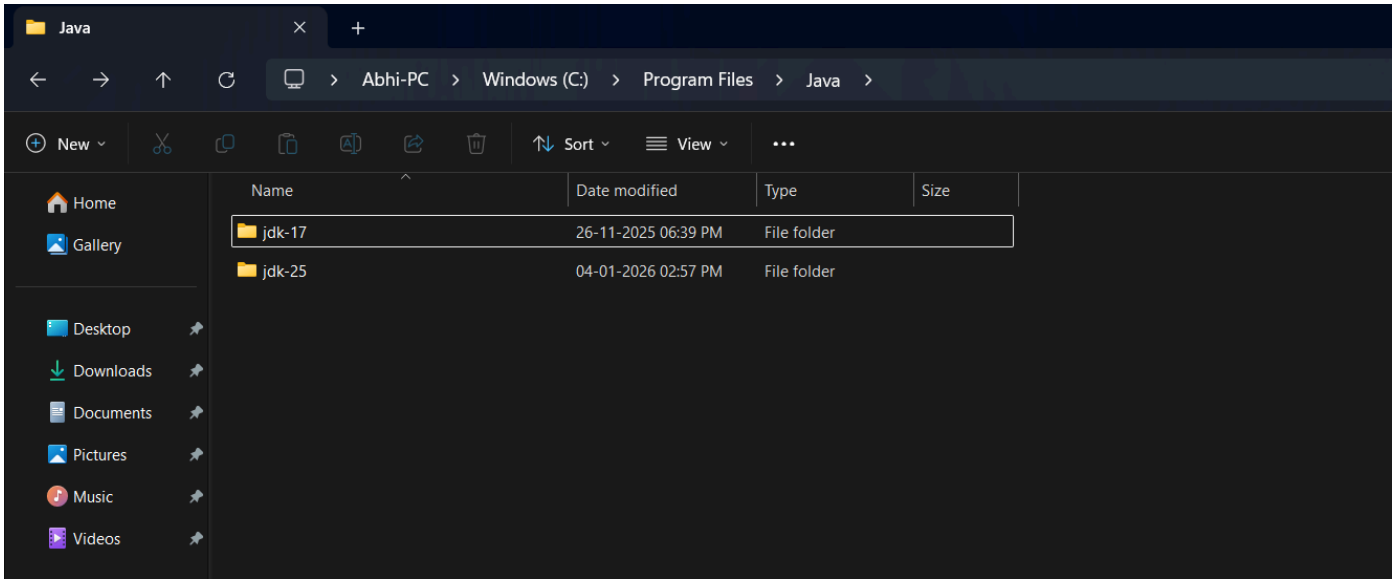
Environment Setup (Java Installation)

Step- 1) Download and Install Java Software

Linux **macOS** **Windows**

Product/file description	File size	Download
x64 Compressed Archive	204.33 MB	https://download.oracle.com/java/25/latest/jdk-25_windows-x64_bin.zip (sha256)
x64 Installer	182.29 MB	https://download.oracle.com/java/25/latest/jdk-25_windows-x64_bin.exe (sha256)
x64 MSI Installer	181.05 MB	https://download.oracle.com/java/25/latest/jdk-25_windows-x64_bin.msi (sha256)

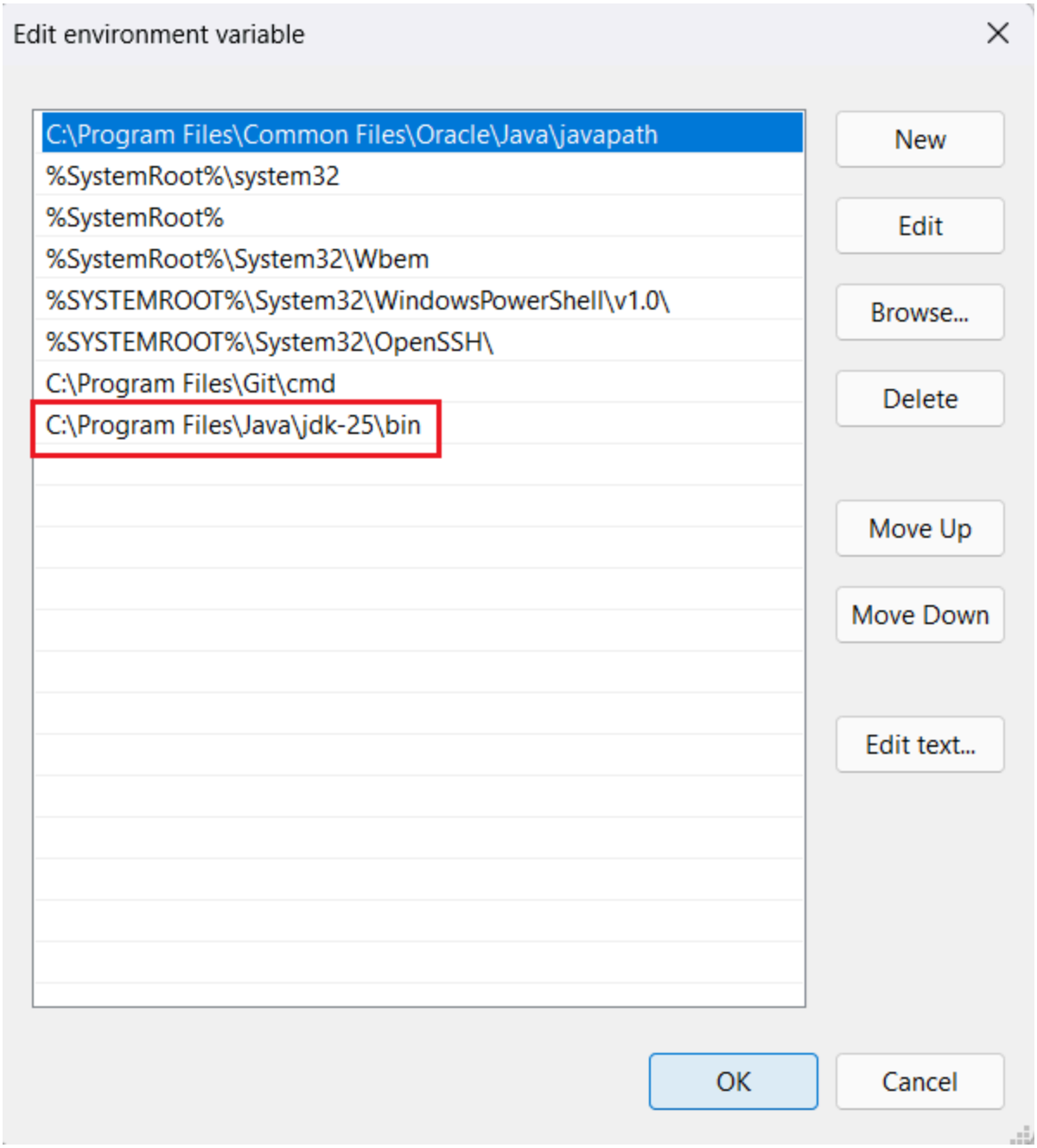
- After installing java, we can see below folder (JDK)



Note: If you installed java first time you'll see only one folder containing the version you installed. (Here Java 17 is previously installed on this system that's why there are two folders)
Note: After installing java software, we need to set PATH for java software to use it.

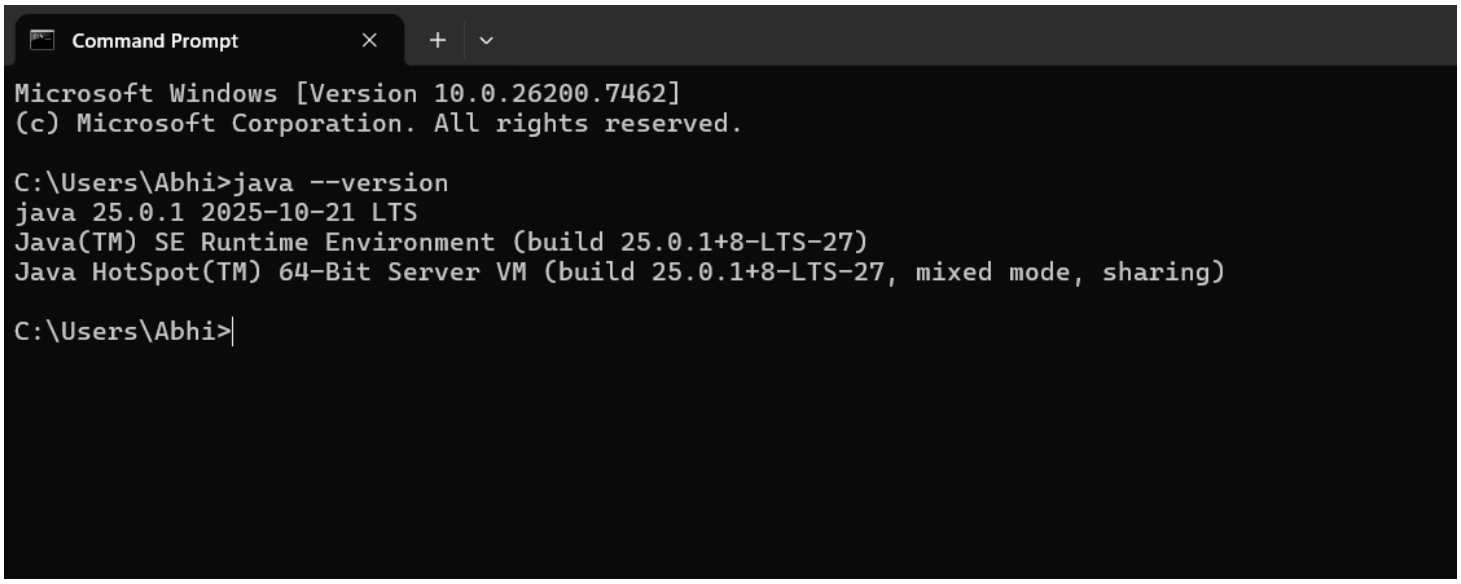
Step-2) Set Path for Java

- Go To Environment Variables
- Go To System Environment Variables
- Edit Path
- Add path up to JDK bin directory
  Path = C:\Program Files\Java\jdk-25\bin



Step-3) Verify PATH Setup

- Open command prompt and execute below command



Note: If you are getting message like above that means, java installation and PATH setup is success.

Assignments (LeetCode)

1. Two Sum
2. Add Digits

Session 2 - Data Types, Variables & Type Casting

Data Types

- Data types are used to specify type of the data
- Data types are divided into 2 categories

```
1) Primitive / Pre-Defined Data Types
      1) Integral Data Types
              - byte
              - short
              - int
              - long
      2) Decimal Data Types
              - float
              - double
      3) Character Data Type
              - char
      4) Boolean Data Type
              - boolean

2) Non-Primitive / Referenced Data Types
      - Arrays
      - Strings
      - Classes
```

Integral Data Types

- Integral data types are used to store numbers without decimal points
- We can store both positive and negative numbers using integral data types

```
// Intergral Data Type
age = 30;
phno = 66868686868;
studentscnt = 40;
balance = - 3000;
```

- We have 4 data types in this category
- For These 4 data types memory & range is different

```
1)  byte   ---->  default value is 0  ---->  1 byte
2)  short       ---->  default value is 0  ---->  2 bytes
3)  int    ---->  default value is 0  ---->  4 bytes
4)  long   ---->  default value is 0l   ---->  8 bytes
```

Decimal Data Types

- Decimal data types are used to store numbers with decimal values

- We can store both postive and negative values

```
// Decimal Data Types
petrol price  = 110.567979;
stockPrice = 334.3279797979797979;
percentage = 9.8;
weight = 55.6;
height = 5.6;
length = 10.2;
```

- In this category we have 2 data types

```
1) float   ----> 4 bytes ---> upto 6 decimal points
2) double  -----> 8 bytes  --> upto 15 decimal points
```

Character Data Type

- Character data type is used to store single character
- Any single character (alphabet / digit / special character) we can store using 'char' data type
- char datatype will occupy 2 bytes of memory
- When we are storing data into 'char' data type single quote is mandatory
- default value is 'u0002'

```
// Character Data Type
gender = 'm';
rank = '1';
```

Note: In C language 'char' will take only 1 byte where as in java 'char' will take 2 bytes

Boolean Data Type

- It is used to store true or false values only
- It will occupy 1 bit memory
- Note: 8 bits = 1 byte
- default value for boolean is false

```java
// Boolean Data Type
isPass = true;
isFail = false;
isMarried = true;
isOdd = true;
isEven = false;
```

How memory allocated ?

- Memory (RAM) is divided into two parts : Stack, Heap
- For primitive data types memory will be inside stack
- For referenced data types memory will be allocated inside heap

Type Casting

- Type casting is the process of converting one type of value into another
- Java supports two types of type casting:

```
Implicit Type Casting (Widening)
Explicit Type Casting (Narrowing)
```

Implicit Type Casting

- Done automatically by the compiler
- Converts a smaller data type to a larger data type
- No data loss and safe conversion

```
byte → short → int → long → float → double
```

```java
int a = 10;
double d = a;   // implicit casting
System.out.println(d); // Output: 10.0
```

Explicit Type Casting (Narrowing)

- Done manually by the programmer
- Converts a larger data type to a smaller data type
- Possibility of data loss

- Requires casting operator `(type)`

```
double → float → long → int → short → byte
```

```java
double d = 10.75;
int i = (int) d;   // explicit casting
System.out.println(i); // Output: 10
```

Wrapper classes

- Java is a Object Oriented Programming Language
- In java we can represent everything in the form of Object
- To represent primitive types data in object format java provided Wrapper classes
- For every primitive type corresponding Wrapper class is available
- All wrapper classes are part of `java.lang` package

```
byte  ----> Byte
short  ---> Short
int  --->   Integer
long  ---> Long
float   ----> Float
double   ----> Double
char   ---> Character
boolean   ---> Boolean
```

```java
int var = 10;
System.out.println(var); // output 10

Integer obj = new Integer(10);
System.out.println(i); // output 10
```

```java
Integer   i1 = new Integer (10) ;
Integer   i2 = new Integer ("20");

Double  d1  = new Double (10.05);
Double  d2 =  new Double ("20.10");

Character c1 = new Character ('a');
Character c1 = new Character ("a");   // invalid
```

Scanner Class

- Scanner class in Java is used to take input from the user
- It is part of the `java.util` package
- It is commonly used to read primitive data types and strings from keyboard

```
nextInt()     --->    Reads an integer
nextFloat()     --->    Reads a float
nextDouble()     --->    Reads a double
next()     --->    Reads a single word
nextLine()     --->    Reads a full line
```

```java
import java.util.Scanner;

public class ScannerExample {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        System.out.print("Enter your name: ");
        String name = sc.nextLine();

        System.out.print("Enter your age: ");
        int age = sc.nextInt();

        System.out.println("Name: " + name);
        System.out.println("Age: " + age);

        sc.close();
    }
}
```

Assignments (LeetCode)

1. Reverse Integer
2. Palindrome Number

Session 3 - Operators in Java

Operators

- Operator is a symbol which performs some operation on operands

```java
int a  = 10 ;
int b = 20 ;
int c = a + b;
```

- We have below operators in java

```
1) Arithmetic Operators
2) Logical Operators
3) Relational Operators
4) Assignment Operators
5) new operator
6) dot (.)  operator
7) ternary operator ( Conditional Operator )
8) Bitwise Operator
```

Arithmetic Operators

- Arithmetic Operators are used to perform Arithmetic Operations (Calculations)

```
1) Addition -----> +
2) Subtraction ---->  -
3) Division ------------>  /  (quotient)
4) Multiplication ----------> *
5) Modulus ---------> %  (reminder)
6) Increment -------> ++
7) Decrement ------> --
```

- Increment (++) is used to increase the value of variable by 1
- Increment is divided into 2 types

```
1) Post Increment   ( a ++ )
2) Pre Increment  ( ++ a)
```

- Decrement (--) is used to decrease the value of variable by 1

- Decrement is divided into 2 types

```
1) Post Decrement ( a -- )
2) Pre-Decrement ( --a )
```

```
// Increment and Decrement Operators

class PostIncrement {
        public static void main(String... args){
                int a = 5;
                System.out.println(a++); // it will print 5 then it will become 6
                a++;  // it will become 7
                System.out.println(a++); // it will print 7 then it will become 8
                System.out.println(a); // it will print 8
        }
}

class PreIncrement {
        public static void main(String... args){
                int a = 5;
                System.out.println ( ++ a );  // it will become 6 then it will prin
                ++ a ;  // it will become 7
                System.out.println(++a);  // it will become 8 then it will print
                System.out.println(a); // it will print 8
        }
}

class PostPreIncrement {
        public static void main(String... args){
                int a = 5;
                int b = ++a  + a++ + a++  + ++a;
                // int b = 6 + 6 + 7 + 9 ==> 28
                System.out.println(b);
        }
}

class Decrement {
        public static void main(String... args){
                int a = 5;
                System.out.println( a -- );  // it will print 5 then it will become
                System.out.println( -- a);  // it will become 3 then it will print
        }
}

class PostPreDecrement {
        public static void main(String... args){
                int a = 5;
                int b =  a-- + --a + a--;
```

```
                    // int b = 5 + 3 + 3
                    System.out.println ( b );
            }
    }
```

Bitwise Operators

- Bitwise operators are used to manipulate individual bit of operands
- Bitwise operators can be applied to integral types

```
Bitwise unary NOT    --->    ~
Bitwise AND    --->    &
Bitwise OR    --->    |
Bitwise exclusive OR    --->    ^
Shift right    --->    >>
Shift right zero fill    --->    >>>
Shift left    --->    <<
```

Relational Operators

- Relations Operators are used to check relation between two Operands

```
>, < , >= , <=, !=, ==
```

Logical Operators

- To check more than one condition then we will use Logical operators

```
AND  ---->  &&
OR -----> ||
NOT -----> !
```

Assignment Operator

- Equals (=) is called as assignment operator
- It is used to assign the value for a variable

```
int a  = 10 ;
```

new operator

- It is used to create the object for a class

```
ClassName refVar = new ClassName( );
```

Note: Creating object means allocating memory in heap area

Dot (.) Operator

- Dot operator is used to access class variables & methods

```
System.out.println ( );
java.lang.String;
java.util.ArrayList;
```

Ternary Operator / Conditional Operator

- Ternary operator is used for decision making

```
// Conditional Operator Syntax
( condition ) ? expression-1 : expression-2
```

- If condition satisfied then expression-1 will execute otherwise expression-2 will execute

instanceof Operator

- It is used to check object reference belong to a class or not

```
String str = "Hi";
if (str instanceof String ) {
        //logic
}
```

Operator Precedence

- Operators at the same level follow associativity rules
- Parentheses `()` have the highest precedence

- Assignment operators have the lowest precedence

```
1. ()                                → Parentheses
2. ++  --                            → increment, decrement
3. ++  --  +  -  !  ~                → Unary operators
4. *  /  %                           → Multiplication, Division, Modulus
5. +  -                              → Addition, Subtraction
6. <<  >>  >>>                       → Shift operators
7. <  <=  >  >=  instanceof          → Relational operators
8. ==  !=                            → Equality operators
9. &                                 → Bitwise AND
10. ^                                → Bitwise XOR
11. |                                → Bitwise OR
12. &&                               → Logical AND
13. ||                               → Logical OR
14. ?:                               → Ternary operator
15. =  +=  -=  *=  /=  %=            → Assignment operators
```

Assignments (LeetCode)

1. Single Number
2. Plus One

Session 4 - Conditional Statements

Conditional Statements

- Execute the code only once based on condition

```
1) simple if
2) if - else
3) nested if
4) if - else - if - else -if - else (if else ladder)
5) switch
```

Simple if

- To execute the statements based on condition

```
// if Syntax
if ( condition ){
    // stmt - 1
    // stmt - 2
    // stmt - 3
}
// or
if (condition )
    //stmt


// Simple if Demo
class SimpleIf{
        public static void main(String... args){
                int a = 100;
                int b = 20;
                if( a > b ) {
                        System.out.println("a is greater than b");
                        System.out.println("Completed");
                }
                System.out.println("Bye");
        }
}
```

if-else

- It is also used to execute the code based on condition

```
// if-else Demo
class IfElseDemo{
        public static void main (String... args){
                int age = 16 ;
                if ( age >= 18 ) {
                        System.out.println("Eligible For Vote") ;
                } else {
                        System.out.println("Not eligible for Vote");
                }
        }
}
```

Nested if-else

```
// Nested if else demo
class NetedIf{
        public static void main(String... args){
                int age = 16 ;
                boolean hasId = true;
                if ( age >= 18 ) {
                        if(hasId){
                                System.out.println("Eligible For Vote");
                        }
                } else {
                        System.out.println("Not eligible for Vote");
                }
        }
}
```

Dangling else Problem

- If we do not use flower brackets to properly indent our code then there is a chance of getting dangling else problem
- Compiler will always associate the `else` part with the closest unmatched `if`
- To avoid dangling else problem we need to use

```
// Dangling else Problem
class NetedIf{
        public static void main(String... args){
                int age = 16 ;
                boolean hasId = true;
                if ( age >= 18 )
                        if(hasId)
                                System.out.println("Eligible For Vote");
                else
                        System.out.println("Not eligible for Vote");
        }
}
```

Requirement :

```
int  a = 20;
if a > 0 -----> display msg as 'a is positive number'
if a < 0 ----> display msg as 'a is negative number'
When above both conditions are failed then display msg as 'a is zero'
```

else-if ladder

```
// else-if ladder syntax
if ( condition_1 ) {
                  // stmt - 1
} else if ( condition_2 ) {
         // stmt - 2
} else  if ( condition_3 ) {
         //stmt - 3
} else {
         //stmt-4
}
```

- if condition_1 is pass then it will execute only stmt-1
- if condition_1 is fail then it will check condition_2
- If condition_2 is pass then it will execute only stmt-2
- If condition_2 is fail then it will check condition_3
- If condition_3 is pass then it will execute only stmt-3
- If condition_3 is fail then directly stmt-4 will be executed

```
// else-if ladder Demo
class IfElseLadderDemo {
         public static void main(String... args){
                  int a = 0;
                  if( a > 0) {
                           System.out.println(" a is positive number ");
                  } else if ( a < 0 ) {
                           System.out.println("a is negative number");
                  } else {
                           System.out.println("a is zero");
                  }
         }
}
```

```java
// Develop a java program to decide role of software engineer based on his/her experie
// 0 - 2  year exp  -----> Associate Engineer
// 3 - 5 years exp -----> Software Engineer
// 6 - 9 years exp -----> Sr.Software Engineer
// 10 - 13 years exp ----> Manager

class RoleFinder {
        public static void main(String... args) {
                int exp = 13;
                if( exp >= 0 && exp <= 2 ){
                        System.out.println("Associate Engineer");
                }else if ( exp >= 3 && exp <= 5 ){
                        System.out.println("Software Engineer");
                }else if(  exp >= 6 && exp <= 9 ){
                        System.out.println("Sr. Software Engineer");
                }else if( exp >= 10 && exp <= 13 ){
                        System.out.println("Manager");
                }else {
                        System.out.println("Role Not Found");
                }
        }
}
```

- In above program we have hardcoded value for the variable
- If we want to test our program with different values we need compile and execute everytime
- To overcome this problem we can read the data from keyboard

switch

- Using switch case we can make decision

```
// switch Syntax
switch ( case ) {

case 1 :  // stmt - 1
break;

case 2 : // stmt - 2
break;

case 3 : // stmt - 3
break;
.....
default : // stmt - default
}
```

```java
// Write a java program to read a number from keyboard. Based on the given number prin
import java.io.*;
class WeekDay {
        public static void main(String... args) {
                int num = 2;

                switch ( num ) {
                        case 1 : System.out.println("Monday");
                                        break;

                        case 2 : System.out.println("Tuesday");
                                        break;

                        case 3 : System.out.println("Wednesday");
                                        break;

                        case 4 : System.out.println("Thursday");
                                        break;

                        case 5 : System.out.println("Friday");
                                        break;

                        case 6 : System.out.println("Saturday");
                                        break;

                        case 7 : System.out.println("Sunday");
                                        break;

                        default : System.out.println("Day not found");
                }
        }
}
```

fall-through concept

```
// Write a java program to read a number from keyboard. Based on the given number print week of
import java.io.*;
class WeekDay {
        public static void main(String... args) {
                int num = 4;

                switch ( num ) {
                        case 1 : System.out.println("Monday");

                        case 2 : System.out.println("Tuesday");

                        case 3 : System.out.println("Wednesday");

                        case 4 : System.out.println("Thursday");

                        case 5 : System.out.println("Friday");

                        case 6 : System.out.println("Saturday");

                        case 7 : System.out.println("Sunday");

                        default : System.out.println("Day not found");
                }
        }
}
```

Conclusion

```
1) 'if' accepts only boolean value (or) boolean expression
2) 'switch' accepts numbers, char & strings (added in java 1.7v)
3) switch will not accept boolean and decimal values
4) switch cases should belongs to same type
5) switch case datatype and switch case input value should belongs to same dataty
6) 'default' case is optional in 'switch case'
7) 'break' keyword is also optional in 'switch case'
```

Switch Advancements (Java 14+)

- Works as an expression (returns a value)
- Uses arrow syntax (`->`)

- No need for `break`

```java
// switch Advancements
int day = 6;

String type = switch (day) {
    case 1, 2, 3, 4, 5 -> "Weekday";
    case 6, 7 -> "Weekend";
    default -> "Invalid";
};
```

Assignments (LeetCode)

1. Number of Steps
2. Power of Two

Session 5 - Loops (for, while, do-while)

Loops

- Loops are used to execute statements repeatedly
- In java we have below loops

```
1) while loop
2) do-while loop
3) for loop
4) for-each loop
```

while loop

- While loop is used to execute statements until condition is true
- while loop is called as conditional based loop
- If condition is true then loop statements will execute otherwise loop will be terminated

```
// while loop syntax
while ( condition ){
        //stmts
}


// Write a java program to print numbers from 1 to 10 using while loop
class WhileDemo {
        public static void main (String... args){
                int i = 1;
                while ( i <= 10 ){
                        System.out.println(i);
                        i++;
                }
        }
}
```

do-while loop

- It is used to execute statements first then it will check the condition
- do-while is also called as conditional based loop only

```
// do-while loop syntax
do{
        //stmts
} while (condition );


// Write a java program to print numbers from 1 to 10 using do-while loop
class DoWhile{
        public static void main(String... args){
                int i = 1;
                do {
                        System.out.println(i);
                        i++;
                } while (i <= 10);
        }
}
```

Q) What is the difference between while and do-while ?

while ==> It will check the condition first then it will execute the statements

do-while ==> It will execute statement first then it will check condition.

Note: Even if condition is not satisfied our statement will execute once.

for loop

- It is used to execute statements multiple times
- For loop is called as Range based loop

```
// for-loop syntax
for ( initialization ; condition ; increment / decrement ) {
        //stmts
}


// Write java program to print numbers from 1 to 10 using for loop
class ForLoop {
        public static void main(String... args) {
                for ( int i = 1 ; i <= 10 ; i++ ) {
                        System.out.println(i);
                }
        }
}
```

break and continue

break

- It is used to come out from switch case and from loops

```
// break demo
class Break {
        public static void main(String... args){
                for (int i = 1; i<= 10; i++ ){
                        if (i >= 5 ){
                                break;
                        }
                        System.out.println(i);
                }
        }
}
```

continue

- It is used to skip one iteration in the loop execution then continue

```
// continue demo
class Continue {
        public static void main(String... args){
                for (int i = 1; i<= 10; i++ ){
                        if(i == 6 ) {
                                continue;
                        }
                        System.out.println(i);
                }
        }
}
```

Infinite Loop

- A loop which does not have any termination (keeps running forever)

```
// Infinite loop
while(true){
        System.out.println("Hello World !!");
}


int i = 10;
while(i == 10){
        System.out.println("Hello World !!");
}
```

Nested Loops

- A loop which contains another loop inside its body

```
// Writing one loop inside another loop is called as Nested loop
        for ( int i = 1; i <= 5 ; i++ ){
                for ( int j = 1; j< = 5; j++){
                }
        }
```

```
// Write a java program to print below pattern using loops
*
* *
* * *
* * * *
* * * * *

class NestedLoop {
        public static void main(String... args){
                for ( int i = 1; i <=5 ; i++ ){
                        for ( int j = 1; j <= i ; j++ ){
                                System.out.print("*");
                        }
                        System.out.println();
                }
        }
}


// Write a java program to print below pattern
1
1 2
1 2 3
1 2 3 4
1 2 3 4 5

class NestedLoop {
        public static void main(String... args){
                for ( int i = 1; i <=5 ; i++ ){
                        for ( int j = 1; j <= i ; j++ ){
                                System.out.print(j);
                        }
                        System.out.println();
                }
        }
}
```

Assignments (LeetCode)

1. Fizz Buzz
2. Count Primes

Session 6 - Arrays (1D & 2D Arrays)

Arrays

- It is a referenced data type
- It is used to store multiple values
- In Arrays, size is fixed (we can't change the size in runtime)
- Arrays supports only homogeneous elements (same type of elements)
- Array is a container which is used to store collection of elements with same data type

```
// Array Declaration
datatype [ ]  variableName;
dataype variableName [ ] ;
datatype  [ ]variableName ;

// Array Creation
variableName = new datatype [ size ] ;
datatype[ ] variableName = new datatype [ size ] ;
int[ ]  arr = new int [ 5 ];
```

Note: At the time of creating the array the size is mandatory

- Array Size represents how many values we can store into Array
- Array will store the data based on indexes
- Array index always will start from zero

```java
// Array Demo
class ArrayDemo {
        public static void main(String... args){
                int [ ]  arr  = new int [ 3 ] ;

                arr[0] = 100;
                arr[1] = 101;
                arr[2] = 102;

                System.out.println ( arr [ 0 ] ) ;
                System.out.println ( arr [ 1 ] ) ;
                System.out.println ( arr [ 2 ] ) ;
        }
}
```

- We can find size of the array using length property

```java
// Array length Property
class ArrayDemo {
        public static void main(String... args){
                int [ ]  arr  = new int [ 5 ] ;
                arr[0] = 100;
                arr[1] = 101;
                arr[2] = 102;

                System.out.println ( arr.length ) ;
        }
}
```

```java
class ArrayDemo {
        public static void main (String... args) {
                int arr[ ]  = new int [ 3 ] ;

                arr[0] = 100;
                arr[1] = 101;
                arr[2] = 102;

                for( int i = 0 ; i < arr.length ; i ++ ) {
                        System.out.println ( arr [i] );
                }
        }
}
```

```java
class ArrayDemo {
        public static void main(String... args) {
                boolean [ ]  arr  = new boolean [ 3 ] ;

                arr [2] = true;

                for( int i = 0 ; i < arr.length ; i ++ ) {
                        System.out.println ( arr [i] );
                }
        }
}
```

```java
class ArrayDemo {
        public static void main(String... args) {
                int arr [ ]  = { 101, 102, 103, 104 } ;
                for ( int i = 0; i < arr.length; i++){
                        System.out.println(arr[i]);
                }
        }
}
```

```
class ArrayDemo {
        public static void main(String... args) {
                int arr [ ]  = { 101, 102, 103, 104 } ;
                arr [ 0 ] = 200 ;  // it will update 0th index value from 101 to 20
                arr [ 1 ] = 300 ;  // it will update 1st index value from 102 to 30
                System.out.println ( arr [ 101 ] ) ;  // AIOBE ( 101 index not avai
        }
}
```

Note: Array size should be positive integer only

- Decimal value can't be used for Array Size
- Negative value also can't be used for Array Size

for-each loop

- for-each loop is used to iterate arrays and collections

```
// for-each loop example
class ArrayDemo {
        public static void main(String... args) {
                int arr [ ]  = { 101, 102, 103, 104 } ;
                for (int temp : arr){
                        System.out.println(temp);
                }
        }
}
```

Types of Arrays

- Arrays are divided into 2 types

```
// 1) Single Dimensional Array
        int arr[ ] =  new int[size];
// 2) Multi Dimensional Array
        int arr[ ][ ] = new int[rowsSize][columnsSize];
```

2D Arrays

- 2D arrays are having rows and columns
- Each row of 2D array is a 1D array

```
// Multi-Dimensional Arrays Demo
class TwoDimensionArray{
        public static void main(String... args){
                int arr [ ] [ ] = new int [ 2 ] [ 2 ];
                arr [0] [0] = 100 ;
                arr [0] [1] = 200;
                arr [1] [0] = 300;
                arr [1] [1] = 400;
                for (int i = 0; i < arr.length ; i ++ ){
                        for ( int j = 0; j <arr.length ; j ++ ){
                                System.out.println(arr[i][j]);
                        }
                }
        }
}


// Iterating 2D array using for-each loop
public class Demo {
        public static void main(String[] args) {
                int arr[][] = { { 100, 200 }, { 300, 400 } };

                for (int[] ar : arr) {
                        for(int x : ar) {
                                System.out.println(x);
                        }
                }
        }
}
```

```java
// Converting Array to String
import java.util.Arrays;

public class Demo {
	public static void main(String[] args) {
		int arr[][] = { { 100, 200 }, { 300, 400 } };

		for (int[] temp : arr) {
			System.out.println(Arrays.toString(temp));
		}
	}
}
```

```java
// Add Two Matrix
import java.util.Scanner;

public class MatrixAddition {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        System.out.print("Enter number of rows: ");
        int rows = sc.nextInt();

        System.out.print("Enter number of columns: ");
        int cols = sc.nextInt();

        int[][] matrix1 = new int[rows][cols];
        int[][] matrix2 = new int[rows][cols];
        int[][] result = new int[rows][cols];

        System.out.println("Enter elements of first matrix:");
        for(int i = 0; i < rows; i++) {
            for(int j = 0; j < cols; j++) {
                matrix1[i][j] = sc.nextInt();
            }
        }

        System.out.println("Enter elements of second matrix:");
        for(int i = 0; i < rows; i++) {
            for(int j = 0; j < cols; j++) {
                matrix2[i][j] = sc.nextInt();
            }
        }

        // Adding matrices
        for(int i = 0; i < rows; i++) {
            for(int j = 0; j < cols; j++) {
                result[i][j] = matrix1[i][j] + matrix2[i][j];
            }
        }

        System.out.println("Result :");
        for(int i = 0; i < rows; i++) {
            for(int j = 0; j < cols; j++) {
                System.out.print(result[i][j] + " ");
            }
```

```
            System.out.println();
        }

        sc.close();
    }
}
```

Session 7 - Strings & StringBuilder

Strings

- String is a pre-defined class available in `java.lang` package
- String is used to store group of characters

```
String name = "abc" ;
```

- String is immutable in java (can't be modified)
- We can create String object in 2 ways

```
// approach - 1  (string literal)
String  name = "arpit";

// approach - 2 (using new operator)
String str = new String ("arpit");
```

String Constant Pool

- It is special memory in JVM to store String objects
- It will not allow us to create duplicate objects

```
String s1 =  "hi" ;
String s2 = "hi" ;
```

- s1 and s2 objects are having same content hence only one object will be created and two variables will be pointing to same object.

```java
class StringDemo {
        public static void main(String[ ] args) {
                String s1 =  "hi" ;
                String s2 = "hi" ;

                if ( s1 == s2 ) {
                        System.out.println (" Both are same ");
                }else {
                        System.out.println(" Both are not same" );
                }
        }
}
```

- If we create String objects using `new` operator always new object will be created in Heap area.

```java
class StringDemo {
        public static void main(String[ ] args) {
                String s1 =  new String ("hello") ;

                String s2 = new String ("hello") ;

                if ( s1 == s2 ) {
                        System.out.println (" Both are same ");
                }else {
                        System.out.println(" Both are not same" );
                }
        }
}
```

Note: In Strings == will compare address of the objects not content.

String class Methods

charAt ( )

- To get a character based on given index

```java
String s1 = "arpit";
System.out.println(s1.charAt(0));
```

length ( )

- To get size of string (no.of characters available in String)

```
String s1 = "arpit";
System.out.println(s1.length( ) );
```

concat ( )

- To join two strings (appending)

```
String s1 = "arp";
String s2 = "it";
String s3 = s1.concat(s2);
// String s4 = s1 + s2 ;
System.out.println(s3);
```

equals ( )

- To compare content of two Strings

```
String s1 = "hi";
String s2 = "hello";
System.out.println( s1.equals(s2) );
```

Note: In Strings, == operator will compare address of string objects where as `equals()` method will compare content of the objects.

toUpperCase ( )

- To convert string to uppercase string

```
String s1 = "hi"
Sytem.out.println(s1.toUpperCase());
```

toLowerCase ( )

- To convert String to lowercase String

```
String s1 = "Hello"
Sytem.out.println(s1.toLowerCase());
```

indexOf ( )

- To get first occurrence of char

```
String s1 = "arpit";
System.out.println(s1.indexOf('a'));
```

lastIndexOf ( )

- to get last occurrence of char

```
String s1 = "Hello"
System.out.println(s1.lastIndexOf('l'));
```

Note: If given char is not available then it will return '-1'

substring ( )

- It is used to get some part of the string. It will take start index & end index.

```
// start-index : inclusive
// end-index : exclusive
System.out.println(s1.substring(0,5));
```

Note: If we don't give end index, it will print from start index to last index.

valueOf ( )

- It is used convert any type value into String type

```
int a = 10 ;
int b = 20 ;

a + b   ===> 30

String s1 = "10";
String s2 = "20";

s1 + s2  ==> "1020"

String.valueOf (a) + String.valueOf (b) ==> 1020
"10" + "20" ==> 1020
```

Note: valueOf ( ) is a static method in String class. Static methods will be called using classname. Non-Static methods will be called using Object.

trim ( )

- It is used to remove starting and ending spaces of String

```
String str = "     hello    ";
str.trim ( );
```

toString ( )

- It is used to convert object into string format.

toCharArray ( )

- This method is used to convert String to char array

```
String s1 = "java";
char arr[ ]  = s1.toCharArray ( );
```

StringBuffer

- It is a predefined class available in `java.lang` package
- It is also used to store group of characters
- StringBuffer is a mutable object (we can modify the content)
- StringBuffer is thread-safe (only one thread can access at a time)

```
StringBuffer sb  = new StringBuffer("hello");


// StringBuffer example
class SBDemo {
        public static void main (String[ ] args){
                StringBuffer sb1 = new StringBuffer("hi");
                sb1.append(" hello");
                System.out.println(sb1);

                StringBuffer sb2 = new StringBuffer("java");
                System.out.println(sb2.reverse());
        }
}
```

StringBuilder

- StringBuilder is a predefined class available in `java.lang` package
- This class introduced in JDK 1.5v
- StringBuilder objects are mutable (content can be modified)
- StringBuilder objects are not thread safe (Multiple threads can access at a time)

```
StringBuilder sb = new StringBuilder("java");
sb.length ( );
sb.append("program");


class BuilderDemo {
        public static void main (String[ ]  args){
                StringBuilder sb =  new StringBuilder("java") ;
                System.out.println(sb.length());

                sb.append(" program");

                System.out.println(sb);
                System.out.println(sb.length());
        }
}
```

Q) What is the difference between String, StringBuffer & StringBuilder ?

String -> Immutable --> Jdk 1.0

StringBuffer -> Mutable + Thread-Safe --> Jdk 1.0

StringBuilder -> Mutable + Not-Thread-Safe --> Introduced in jdk 1.5v

Session 8 - Classes & Objects

Class

- Class is a plan or model or template
- Class is a blue print of object
- Class is used to declare variables & methods
- Once class is created then we can create any no of objects for a class
- `class` keyword is used to create Classes in java

```
// class sytax
class ClassName {
        // variables
        // methods
}
```

Note: Classes will not exist physically.

Object

- Any real-world entity is called as Object
- Objects exist physically
- Objects will be created based on the Classes
- Without having the class, we can't create object (class is mandatory to create objects)
- Object creation means allocating memory in Heap
- `new` keyword is used to create the objects

```
// obj creation syntax
ClassName   refVariable = new ClassName();
```

```
User  u1  = new User();
User u2 = new User() ;
```

Variables

- Variables are used to store the data

```
int a  = 10 ;

User u1 = new User ( );
Student s1 = new Student ( );
```

- Variables are divided into 3 types

```
1) Global Variables / instance variables / non-static variables
2) static variables
3) local variables
```

Instance Variables

- Variables which are declared inside the class and outside the method are called as instance variables
- instance variables can be accessed by all the methods available in the class thats why these are called as Global Variables
- Instance variables are called as object variables
- When we create the object, then only memory will be allocated for instance variables
- Every Object will maintain its own copy of the instance variable

Note: If we create 2 objects, then 2 times memory will be allocated for instance variables

```java
public class User {
        int age;

        public static void main(String[] args) {
                User raju = new User();
                raju.age = 20;
                System.out.println(raju.age);

                User rani = new User();
                rani.age = 25;
                System.out.println(rani.age);
        }
}
```

Static Variables

- The variables which are declared inside the class and outside the method with `static` keyword are called as static variables
- Static variables are class level variables
- When class is loaded into JVM then immediately memory will be allocated for static variables
- Memory will be allocated for static variables only once when the class is loaded into JVM
- All objects of the class will maintain same copy of the static variables
- Static variables we will access using classname

```java
public class Student {
        String name;
        String email;
        long phno;
        static String institute;

        public static void main(String[] args) {
                Student.institute = "TeachToTech";

                Student ankit = new Student();
                ankit.name = "Ankit";
                Student goutham = new Student();
                goutham.name = "Goutham";
        }
}
```

When to declare variable as static or non-static ?

- If we want to store different value based on object then use instance variable
- If we want to store same value for all objects then use static variable

Local Variables

- The variables which are declared inside the method or constructor or block are called as Local Variables
- If we declare a variable with in the method, then that variable can be used / accessed only with in that method
- Before using local variables, we have to initialize them
- If we create a variable with in the method, memory will be allocated for that variable when that method is executing. After that method execution completed, local variable will be deleted from memory

```
class Demo {
        public static void main(String[ ] args){
                int a = 20;
                int b = 20;
                System.out.println(a);
                System.out.prinltn(b);
        }
}
```

Methods

- Methods are used to perform some operation
- In a class we can write any no of methods including main method
- If we want to execute our method then we have to call our methods from main ( ) method

```
// metod syntax
returnType methodName (param1, param2, para3 ..... paramN) {
        //logic
        return value;
}
```

```java
// Write a method to print a msg on console
void hello ( ) {
        System.out.println("Hello");
}


// Take 2 numbers as input and return sum as ouput
int  add ( int a, int b ) {
        int c = a + b ;
        return c;
}


// take 2 names as input, concat them and print on console
void  fullname (String fname, String lname) {
        String name = fname + lname;
        System.out.println(name);
}


// take person age as input, if person age >=18 then return true else return false
boolean  check (int age ){
        if ( age >= 18 ) {
                return true;
                } else {
                return false;
        }
}
```

Types of Methods

- Methods are divided into 2 types

```
1) instance methods   ---> Object level methods
2) static methods ----> Class level method
```

- instance method will be called by using Object
- static method will be called by using Class

```java
public class Student {
        public static void main(String[] args) {
                Student s1 = new Student();
                s1.hello();
                Student.greet();
        }

        void hello() {
                System.out.println("Hello My Friend...");
        }

        static void greet() {
                System.out.println("Good Evening..");
        }
}


// Write a java method to print data available in the Student object
class Student {
        int id;
        String name;
}

public class Driver {
        public static void main(String[] args) {
                Driver d = new Driver();

                Student s = new Student();
                s.id = 101;
                s.name = "raju";

                d.print(s);
        }
        void print(Student s) {
                System.out.println(s.id + "  " + s.name);
        }
}
```

```
//  Take employee class with id and salary as properties
//  Take Driver class and write the method to print employee object data & call the pr
class Employee {
        int id;
        double salary;
}

public class Driver {
        public static void main(String[] args) {
                Driver d = new Driver();

                Employee e = new Employee();
                e.id = 101;
                e.salary = 15000.00;

                d.print(e);
        }

        void print(Employee e) {
                System.out.println(e.id + "--" + e.salary);
        }
}
```

Session 9 - Constructors & Access Specifiers

Constructor

- Constructor is a special method in java which is used to initialize instance variables
- Constructor name should be same as class name
- Constructor shouldn't have any return type (not even void)

```
class Demo {
        Demo ( ) {
                // logic
        }
}
```

- **Note: At the time of object creation our class Constructor will be executed. Constructor is mandatory to create the object.**
- **Note: If we don't write the constructor in class, then java compiler will add one default constructor to our class.**
- **Note: If we write constructor in the class, then compiler will not add any constructor.**
- Constructors are divided into 2 types

```
1) Zero Param Constructor / Default Constructor
2) Parameterized Constructor
```

```java
// Constructor without parameters
class Student {
        Student ( ) {
                ...
        }
}


// Constructor which contains 1 or more paramters
class Student {
        Student (int i, int b ) {
                ...
        }
}


class Employee {
        public Employee(String str) {
                System.out.println(str + ", Constructor Executed");
        }

        public static void main (String[] args) {
                Employee emp = new Employee("Hi");
        }
}
```

this keyword

- this is a predefined keyword in java
- It is used to represent current class object

```java
class Employee {
        String name;
        float salary;

        public Employee(String name, float salary) {
                this.name = name;
                this.salary = salary;
                System.out.println(this.name + "--" + this.salary);
        }

        public static void main(String[] args) {
                Employee emp = new Employee("Raju", 55000.00f);
        }
}
```

Constructor Overloading

- Writing more than one constructor with different parameters is called as Constructor Overloading

```java
// Constructor Overloading
class Employee {
        Employee(int id){
                // logic
        }

        Employee(double d){
                // logic
        }

        Employee(int id, double d){
                // logic
        }
}
```

Constructor Chaining

- When one constructor calls another constructor it is called constructor chaining

```java
// Constructor chaining
class Employee {
        int id;
        double salary;
        String role;

        Employee() {
                this(100, 10000.0, "Teacher"); // constructor chaining
        }

        Employee(int id, double salary, String role) {
                this.id = id;
                this.salary = salary;
                this.role = role;
        }

        public static void main(String[] args) {
                Employee e = new Employee();
                System.out.println(e.toString());
        }
}
```

Access Specifiers

- These are used to specify accessibility for our classes, constructors, variables & methods
- In java we have below 4 access specifiers / modifiers

```
1) public
2) private
3) protected
4) default
```

public

- It is used to specify global access for classes, variables, methods and Constructors

```
-> We can take class as public
-> We can take constructor as public
-> We can take variables as public
-> We can take methods as public
```

Note: public means anybody can access from inside and outside the class also.

private

- It is used to specify local access (with in the class). private variables, private methods, private constructors can't be accessed outside of the class.

```
-> We can't use private for classes  (not allowed)
-> We can take variables as private
-> We can take constructor as private (no body can create obj from outside c
-> We can take method as private (no body can call our method from outside c
```

protected

- Protected members can be accessed in same package & its sub classes

default

- default members can be accessed in same package. When we don't specify any modifier then it comes under default modifier

```
private  -->  default  -->  protected  -->  public
```

Note: only public and default modifiers are allowed for classes. private and protected are not allowed.

Session 10 - Inheritance & Polymorphism

OOPS

- Object-oriented programming (OOPs) is a methodology that simplifies software development and maintenance by providing some rules
- OOPs Principles are

```
Encapsulation
Inheritance
Polymorphism
Abstraction
```

Encapsulation

- It is used for data hiding
- We will combine variables & methods as one single unit using Class

```java
public class Account {
        private int accNum;
        private String name;

        public void setAccNum(int accNum) {
                this.accNum = accNum;
        }

        public int getAccNum() {
                return this.accNum;
        }

        public void setName(String name) {
                this.name = name;
        }

        public String getName() {
                return this.name;
        }
}

public class Test {
        public static void main(String[] args) {
                Account obj = new Account(); // obj creation
                obj.setAccNum(797979);
                obj.setName("Arpit");

                int accNum = obj.getAccNum();
                String name = obj.getName();
                System.out.println(accNum + "--" + name);
        }
}
```

Inheritance

- The process of extending the properties from one class to another class is called as Inheritance
- To extend the properties we will use `extends` keyword

- From which class we are extending the properties that class is called as `Parent` or `Super` or `Base` class
- The class which is extending the properties is called as `Child class` or `Sub class` or `Derived` class
- By using inheritance we can achieve code re-usability

```
public class Person {
        int id;
        String name;
}

public class Student extends Person {
        int rank;

        public static void main(String[] args) {
                // child class object creation
                Student s = new Student();
                s.rank = 1;

                // accessing parent class properties using child cls obj
                s.id = 101;
                s.name = "Raju";
                System.out.println(s.id + "--" + s.name + "--" + s.rank);
        }
}
```

Note: In above example Person class is acting as Parent class and Student class is acting as Child class.

```java
public class User {
        int id;
        String name;

        void m1() {
                System.out.println("Parent class :: m1() method called");
        }
}

public class Employee extends User {
        void m2() {
                System.out.println("Child class :: m2() method called");
        }

        public static void main(String[] args) {
                // creating object for child class
                Employee emp = new Employee();

                // calling parent class method
                emp.m1();

                // calling child class method
                emp.m2();
        }
}
```

Note: Whenever we create child class object, then first it will execute parent class zero-param constructor and then it will execute child class constructor.

Q) Why Parent class constructor is executing first ?

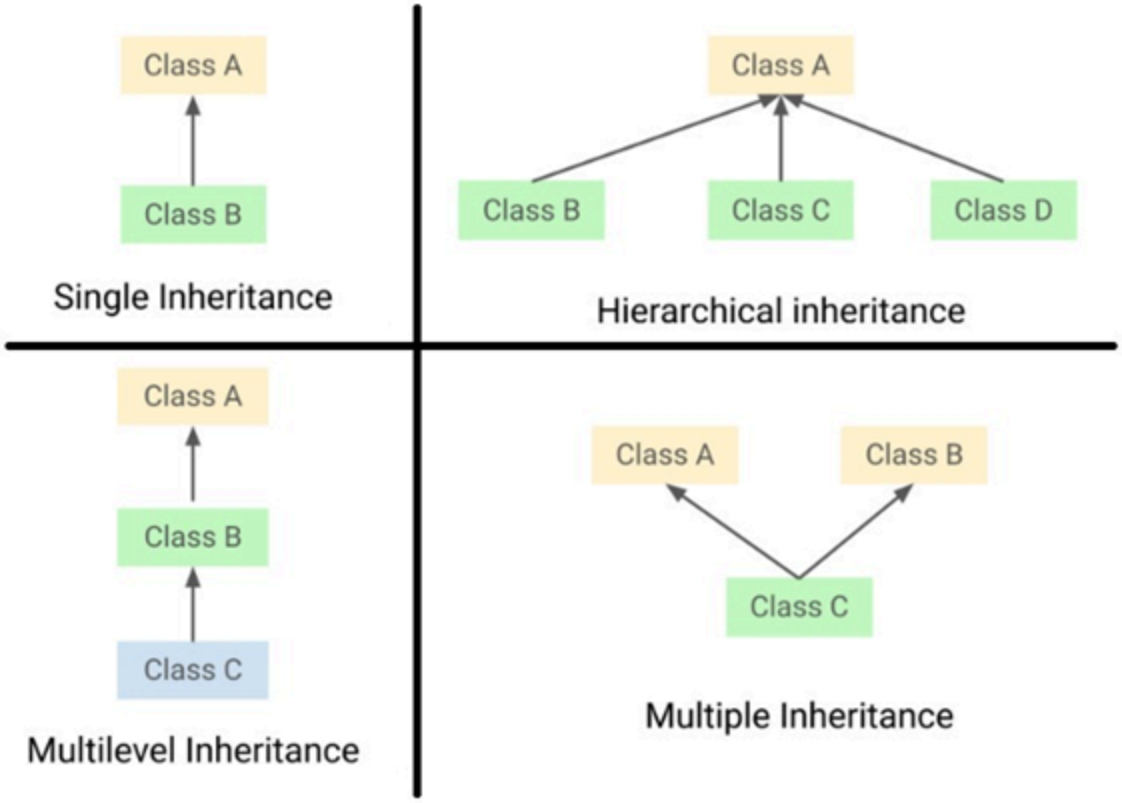
- Child should be able to access parent properties hence parent constructor will execute first to initialize parent class properties.

```java
// inhertience w.r.t to constructors
public class User {
        int id;
        String name;

        public User() {
                System.out.println("Parent class :: 0-Param constructor called ");
        }
}

public class Employee extends User {
        double salary;

        public Employee() {
                System.out.println("Child class :: 0-Param Constructor called");
        }

        public static void main(String[] args) {
                // creating object for child class
                Employee emp = new Employee();

                // initializing parent class properties using child obj
                emp.id = 101;
                emp.name = "John";

                // initialing child class properties using its own obj
                emp.salary = 4500.00;
                System.out.println(emp.id + "--" + emp.name + "--" + emp.salary);
        }
}
```

Types of Inheritance

- Inheritance is divided into multiple types

```
1) Single Level
2) Multi Level
3) Multiple --------> Not supported by Java due to Ambiguity problem
4) hierarchical
```



Single level : Class B extends Class A

Multi Level : Class C extends Class B extends class A

Multiple Inheritance : If one child having more than one parent (Java doesn't support to avoid ambiguity)

Hierarchical : If one parent having multiple child

Why Java Does Not Support Multiple Inheritance ?

- Java does **not support multiple inheritance of classes** to avoid the **Diamond Problem** and reduce ambiguity

```java
public class A {
        public void m1(){
                System.out.println("A :: m1()");
        }
}

public class B {
        public void m1(){
                System.out.println("B :: m1()");
        }
}

// It will not execute, not allowed in Java
public class Demo extends A, B {
        public static void main(String[] args) {
                Demo d = new Demo();
                d.m1(); // which m1() will call
        }
}
```

Method Execution Flow w.r.t Inheritance

When we call a method using Object, first it will check in current class for that method, if available it will call that method directly. If method not available in current class then it will check in parent class (It can be direct or indirect parent). If parent having that method then it will call parent class method. If parent class also doesn't have method then it will throw Exception.

Note: In inheritance always priority will be given for Child class / Sub class object. If child class doesn't contain that method then priority will be given to Parent class method.

super keyword

- super keyword is used to access parent class properties in child class
- It is generally used when parent and child have same property name

```java
// super keyword
class Parent {
    int x = 10;
}

class Child extends Parent {
    int x = 20;

    void display() {
        System.out.println("Child x: " + x);
        System.out.println("Parent x: " + super.x);
    }
}

public class Test {
    public static void main(String[] args) {
        Child obj = new Child();
        obj.display();
    }
}
```

Polymorphism

- If any object is exhibiting multiple behaviour based on the situation then it is called as Polymorphism
- Polymorphism is divided into 2 types

```
1) Static polymorphism / Compile-time Polymorphism
              Ex: Overloading

2) Dynamic polymorphism / Run-time Polymorphism
              Ex: Overriding
```

Method Overloading

- The process of writing more than one method with same name and different parameters is called as Method Overloading

```java
public class Calculator {
        void add (int i, int j) {
                System.out.println("Sum from 1st method :" + (i + j));
        }

        void add (int i, int j, int k) {
                System.out.println("Sum from 2nd method : " + (i + j + k));
        }

        public static void main(String[] args) {
                Calculator c = new Calculator();

                c.add(10, 20);
                c.add(10, 20, 30);
                c.add(10, 20, 30, 40);   // invalid
        }
}
```

- When methods are performing same operation then we should give same name hence it will improve code readability.

```java
substring (int start)
substring(int start, int end)
```

Method Overriding

- The process of writing same methods in Parent class & Child class is called as Method Overriding

```
class Parent {
        void m1( ) {
                // logic
        }
}

class Child extends Parent {
        void m1 ( ){
                //logic
        }

        public static void main(String[] args){
                Child c = new Child ( );
                c.m1 ( );
        }
}
```

- **Note: When we don't want to execute Parent method implementation, then we can write our own implementation in child class using method Overriding.**

```java
// Write a program on Method Overriding
class RBIBank {
        boolean checkElgibility() {
                // docs verification logic
                return true;
        }

        double getHomeLoanRofi() {
                return 10.85;
        }
}

public class SBIBank extends RBIBank {
        // overriding parent method to give my own rofi
        double getHomeLoanRofi() {
                return 12.85;
        }

        public String applyHomeLoan() {
                boolean status = checkElgibility(); // parent method
                if (status) {
                        double homeLoanRofi = getHomeLoanRofi(); // child method
                        String msg = "Your loan approved with RI as ::" + homeLoan
                        return msg;
                } else {
                        return "You are not elgible for home loan";
                }
        }

        public static void main(String[] args) {
                SBIBank bank = new SBIBank();
                String msg = bank.applyHomeLoan();
                System.out.println(msg);
        }
}
```

Dynamic Method Dispatch

- Dynamic Method Dispatch is a mechanism by which a call to an overridden method is resolved at runtime

```java
// Dynamic method dispatch
class Parent {
    void show() {
        System.out.println("Parent method");
    }
}

class Child extends Parent {
    void show() {
        System.out.println("Child method");
    }
}

public class Test {
    public static void main(String[] args) {
        Parent obj = new Child();
        obj.show(); // Dynamic method dispatch
    }
}
```

final keyword

- final is a reserved keyword in java
- We can use final keyword at 3 places

```
1) class level
2) variable level
3) method level
```

- final classes can't be inherited. We can't extend properties from final classes. Final classes are immutable

```java
public class User extends String { // invalid because String is final class
         // logic
}
```

- final variables are nothing but constants
- Final variable value can't be modified

```java
public final double pi = 3.14;
pi = 4.32; // invalid
```

- final methods can't be overridden. We can't override final methods

```java
class Parent {
        final void m1() {
                System.out.println("Parent :: m1()");
        }
}

class Child extends Parent {
        void m1() { // invalid final method can not overridden
                System.out.println("Child :: m1()");
        }
}
```

Session 11 - Abstraction & Interfaces

Abstraction

- The process of hiding un-necessary part and showing only useful part is called as Abstraction
- We can achieve abstraction using Interfaces & Abstract classes

Method Types

- In java we can write 2 types of methods

```
1) Concrete Method
2) Abstract Method
```

- The method which contains body is called as Concrete method

```java
public void m1( ){
        // logic
}
```

- The method which doesn't contain body is called as Abstract method

```
public abstract void m2( );
```

Note: By using `abstract` keyword we can create abstract methods & abstract classes.

Abstract Classes

- The class which contains both concrete and abstract methods is called as abstract class
- Abstract classes are used to achieve partial abstraction
- We will use `abstract` keyword to represent class as abstract class
- We can write constructor in abstract class but we can't create object
- Abstract classes can have child classes
- When we extend properties from abstract class then it is mandatory to override all abstract methods of that class
- Abstract class constructor will be executed when we create object for child class

```
// Abstract class declared with the abstract keyword
abstract class Animal {
    public abstract void makeSound();

    public void sleep() {
        System.out.println("Animal Sleeping");
    }
}

class Dog extends Animal {
    public void makeSound() {
        System.out.println("Woof Woof");
    }
}

public class Main {
    public static void main(String[] args) {
        Dog myDog = new Dog();
        myDog.makeSound();
        myDog.sleep();
    }
}
```

Interfaces

- Interfaces are used to achieve loosely coupling & abstraction
- Interfaces contains only abstract methods (upto 1.7v of java)
- To create a interface we will use `interface` keyword
- Interface doesn't contain constructor
- We can't create object for the interface
- Once interface is created then anybody can provide implementation for the interface
- Implementing interface means overriding interface abstract methods
- When we are implementing a interface then it is mandatory to implement all abstract methods of that interface
- To implement a interface we will use `implements` keyword

Note: One java class can implement Multiple Interfaces at a time.

```java
// Payment interface
interface Payment {
    void pay(double amount);   // abstract method
}

// CreditCardPayment class implementing Payment interface
class CreditCardPayment implements Payment {
    public void pay(double amount) {
        System.out.println("Credit Card Payment");
    }
}

// UPIPayment class implementing Payment interface
class UPIPayment implements Payment {
    public void pay(double amount) {
        System.out.println("UPI Payment");
    }
}

// Main class
public class Main {
    public static void main(String[] args) {
        Payment p1 = new CreditCardPayment();
        p1.pay(1000);

        Payment p2 = new UPIPayment();
        p2.pay(500);
    }
}
```

Note: Interface reference variable can hold its implementation class object.

```java
// Interface example
public interface Bank {
        public void moneyTransfer();
        public void checkBalance();
}

public class HdfcBank implements Bank {
        public void moneyTransfer() {
                System.out.println("Money Transfer from HDFC....");
        }

        public void checkBalance() {
                System.out.println("Checking Balance from HDFC.....");
        }
}

public class AxisBank implements Bank {
        public void moneyTransfer() {
                System.out.println("Money Transfer from Axis ....");
        }

        public void checkBalance() {
                System.out.println("Check Balance from Axis....");
        }
}

public class BankDemo {
        public static void main(String[] args) {
                Bank b; // reference variable

                b = new AxisBank(); // storing impl obj into ref variable
                b.moneyTransfer(); // axis-bank method will be called
                b.checkBalance(); // axis-bank method will be called

                b = new HdfcBank();  // storing impl obj into ref variable
                b.moneyTransfer(); // hdfc-bank method will be called
                b.checkBalance(); // hdfc-bank method will be called
        }
}
```

- In Interface we can declare variables also, by default they are public static final

```
public interface Bank {
        int rateOfInterest = 12;
}
```

Multiple Inheritance using Interfaces

- In Java we can achieve multiple inheritance using interfaces
- A class can multiple interfaces

```
public interface A {
        public void m1();
}

public interface B {
        public void m1();
}

public class Demo implements A, B {
        public void m1() {
                System.out.println("Demo :: m1()");
        }

        public static void main(String[] args) {
                Demo d = new Demo();
                d.m1(); // Demo m1() will called
        }
}
```

Session 12 - Packages

Packages

- Packages are used to group the classes
- Sun people divided predefined classes into several packages

```
1) java.lang
2) java.io
3) java.util
4) java.sql
```

- `java.lang` package is default package and it is available for all java classes by default
- If we want to use any predefined class which is not part of `java.lang` package then we have to import that class using `import` keyword

```java
// Example of predefined package import
import java.util.Scanner;

public class ScannerExample {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        System.out.print("Enter your name: ");
        String name = sc.nextLine();

        System.out.println("Name: " + name);

        sc.close();
    }
}
```

User Defined Packages

- In our project we will create our own packages to organize project related classes & interfaces
- To create a package we will use `package` keyword
- In java class, package statement should be in first line and class should have only one package statement
- We can create user-defined package like below

```
package com.tcs.aadhar;
```

- Sample package naming convention : `company-name.project-name.module-name`

```
com.ibm.irctc.admin
      AdminLogin.java
      AdminService.java
      Request.java
```

Static Imports

- Static import is used to access static members directly without using class name

```java
// Person class
package com.ibm.aadhar;

public class Person {
        public static void speak() {
                System.out.println("Hi, i am ashok");
        }

        public static void m1() {
                System.out.println("hi, i am from m1()");
        }

        public static void m2() {
                System.out.println("hi, i am from m2()");
        }
}


// Java Program with Normal Import to use Person class
import com.ibm.aadhar.Person;

public class Demo {
        public static void main(String[] args) {
                Person.speak();
                Person.m1();
                Person.m2();
        }
}
```

```java
// java program with static import
import static com.ibm.aadhar.Person.*;

public class Demo {
        public static void main(String[] args) {
                speak();
                m1();
                m2();
        }
}
```

Java Program Execution using CMD

- `javac` command is used to compile the java program

```
javac <filename.java>
```

- After compilation `.class` file will be generated
- `java` command is used to execute the `.class` file

```
java <filename>
```

Classpath

- Classpath is used to locate where our .class files / jar files are available
- If all these .class files are not present in the same directory, then we need to specify the path for all the .class files
- You can add any no of paths to the classpath which are separated by `";"` . You can add the current directory to the classpath by using `"."`
- There are 2 ways to set classpath

```
-cp : to set classpath for a particular command
-classpath : to set classpath for a particular command
```

JAR Files

How to create JAR file

- JAR stands for Java Archive
- JAR contains collection of .class files

```
// Syntax to create a jar file
jar  cf  <filename.jar> <package-name>
```

```
c - create
f - file
```

Using JAR in Another Program

- We can use JAR files to add libraries in our program

```
java -cp <filename.jar> <classname>
```

Session 13 - Exception Handling

Exception

- Un-expected and Un-wanted situation in the program execution is called as Exception
- Exception will disturb normal flow of the program execution
- When Exception occurred program will be terminated abnormally
- To achieve graceful termination we need to handle the exceptions occurred while program executing
- The process of handling Exceptions is called as Exception Handling
- The main aim of Exception Handling to achieve graceful termination of the program
- In java we have so many predefined exceptions

```
ArithematicException
NullPointerException
FileNotFoundException
SQLException
```

Exception vs Error

- Exception can be handled
- Error can't be handled

```
Ex: Assertion Error, Virtual Machine Error
```

Exception Types

- Exceptions are divided into 2 types

```
1) Checked Exceptions
              Ex: IOException, FileNotFoundException, SQLException etc....
2) Un-Checked Exceptions : Will occur at Run time ( Compiler can't identify
              Ex: NullPointerException, ArithematicException etc...
```

Checked Exceptions

- Will be identified at compile time (occurs at run time)

```
Ex: IOException, FileNotFoundException, SQLException etc....
```

Un-Checked Exceptions

- Will occur at Run time (Compiler can't identify these exception)

```
Ex: NullPointerException, ArithematicException etc...
```

Handling Exceptions

- Java provided 5 keywords to handle exceptions

```
1) try
2) catch
3) finally
4) throws
5) throw
```

try

- It is used to keep risky code

```
// Syntax
try {
          // stmts
}
```

catch

- catch block is used to catch the exception which occurred in try block
- To write catch block , try block is mandatory
- One try block can contain multiple catch blocks also

```
// Syntax
try {
          // logic
} catch ( Exception e ){
          // logic to catch exception info
}
```

Note: If exception occurred in try block then only catch block will execute otherwise catch block will not execute.

```java
public class Demo {
        public static void main(String[] args) {
                System.out.println("main( ) method started...");
                try {
                        System.out.println("try block start");
                        String s = null;
                        s.length(); // NPE
                        System.out.println("try block end");
                } catch (Exception e) {
                        System.out.println("catch block");
                        e.printStackTrace();
                }
                System.out.println("main( ) method ended...");
        }
}


// We can write one try block with multiple catch blocks also like below
public class Demo {
        public static void main(String[] args) {
                System.out.println("main( ) method started...");
                try {
                        System.out.println("try block start");
                        String s = "hi";
                        int i = s.length();
                        System.out.println("try block end");
                } catch (ArithmeticException e) {
                        System.out.println("catch block");
                        e.printStackTrace();
                } catch (NullPointerException e) {
                        e.printStackTrace();
                } catch (Exception e) {
                        e.printStackTrace();
                }
                System.out.println("main( ) method ended...");
        }
}
```

Note: Catch blocks order should be child to parent.

finally

- It is used to perform resource clean up activities

```
Ex: file close, db connection close etc....
```

- finally block will execute always (irrespective of the exception)

```java
// java program with try-catch-finally scenario
public class Demo {
        public static void main(String[] args) {
                System.out.println("main( ) method started...");
                try {
                        System.out.println("try block start");
                        int i = 10 / 2;
                        System.out.println("try block end");
                } catch (Exception e) {
                        System.out.println("catch block");
                        e.printStackTrace();
                } finally {
                        System.out.println("finally block");
                }
                System.out.println("main( ) method ended...");
        }
}
```

Q) What is the difference between final, finalize( ) and finally ?
Ans: final : it is a keyword which is used to declare final variables, final methods and final classes.
finalize ( ) : It is predefined method available in Object class, and it will be called by garbage collector before removing unused objects from heap area.
finally : it is a block we will use to execute some clean activities in exception handling.

```
try : it is used to keep our risky code
catch : It is used to catch the exception occurred try block
finally : to execute clean up activities
```

throws

- It is used to hand over checked exceptions to caller method / JVM
- **Note: We can ignore checked exceptions using throws keyword.**

```java
public class Demo {
        public static void main(String[] args) throws FileNotFoundException {
                FileReader fr = new FileReader("abc.txt");
        }
}
```

- **Note: We can ignore all checked exceptions like below**

```java
public class Demo {
        public static void main(String[] args) throws Exception {
                FileReader fr = new FileReader("abc.txt");
                Class.forName("");
        }
}
```

throw

- throw keyword is used to create the exception

```java
throw new Exception("Msg");
```

```java
public class Demo {
        public String getName(int id) throws Exception {
                if (id == 100) {
                        return "raju";
                } else if (id == 101) {
                        return "rani";
                } else {
                        throw new Exception("Invalid Id");
                }
        }

        public static void main(String[] args) throws Exception {
                Demo d = new Demo();
                String name = d.getName(101);
                System.out.println(name);

                String name1 = d.getName(200);
                System.out.println(name1);
        }
}
```

User Defined Exceptions

- In java language we have several pre-defined exception classes

```
IOException
FileNotFoundException
ClassNotFoundException
SQLException
AirthematicException
ArrayNegativeSizeException
NullPointerException
ClassCastException etc...
```

- Based on Project requirement, sometimes we need to create our own exceptions those are called as user defined exceptions

```
InvalidCredentialsExceptions
NoRecordsFoundException
NoDataFoundException
InvalidInputException
```

- To create our own Exception we need to extend the properties from `Exception` or `RuntimeException` class

```java
public class NoDataFoundException extends RuntimeException {
        public NoDataFoundException() {
                // logic
        }

        public NoDataFoundException(String msg) {
                super(msg);
        }
}

public class Demo {
        private String getName(int id) {
                if (id == 100) {
                        return "Raju";
                } else if (id == 101) {
                        return "Rani";
                } else {
                        throw new NoDataFoundException("Invalid Id");
                }
        }

        public static void main(String[] args) {
                Demo d = new Demo();
                d.getName(200);
        }
}
```

JVM Reaction to Exceptions

- When an error (exception) happens, the program stops running normally
- The JVM creates an exception object that stores information about the error
- The JVM looks for a `catch` block to handle the error
- It checks the current method first
- If not found, it goes back to the previous method (this is called stack unwinding)
- It continues checking methods one by one
- If a matching `catch` block is found, the error is handled there
- If no `catch` block is found, the JVM prints the error message and stops the program